

ID2221 Data-Intensive Computing

Lab assignment Week 1

Alexía Margrét Axelsdóttir
Gudný Eva Eiríksdóttir
Kinga Koltai

September 13, 2026

Introduction

This report outlines our design and implementation of a Generic Urban Data Integration Platform. The platform is built on 4 individual data sources, each using different data formats and naming conventions. These heterogeneous datasets should be integrated into a common analytical repository using our reusable data engineering platform. This marks the first of four parts in our continuous lab assignment; building the data platform.

Task 1.

Before implementing the platform, we must first understand the characteristics of the datasets. To do this, we created individual Data Catalogs for each dataset, where we explore the attributes and behavior of the data.

taxi_zone_lookup.csv	
Primary entity	A NYC taxi zone
Primary key	LocationID
Join attributes	LocationID
Temporal attributes	None
Categorical attributes	Borough, Zone, service_zone
Growth	The table acts as a lookup table and is therefore relatively stable, with no predictable growth over time. It will grow if new NYC taxi zones are added.

Table 1: Data Catalog for the Taxi Zone Lookup dataset

weather.csv	
Primary entity	One hourly weather observation
Primary key	year, month, day, hour
Join attributes	year, month, day, hour
Temporal attributes	year, month, day, hour
Categorical attributes	All of the *_source columns
Growth	One new observation entry is added each hour, so the growth of the table is fixed and predictable.

Table 2: Data Catalog for the hourly Weather dataset

hourly_88101_2024.csv	
Primary entity	One sensor measurement of the air quality in a specific location and point in time
Primary key	State Code, County Code, Site Num, POC, Date Local, and Time Local
Join attributes	State Code, Country Code, Site Num, Method Code, Date GMT, Time GMT
Temporal attributes	Date Local, Time Local, Date GMT, Time GMT, Date of Last Change
Categorical attributes	State Code, State Name, County Code, County Name, Parameter Code, Parameter Name, Method Code, Method Name, Site Num, POC, datum, Units of Measure, Qualifier, Method Type
Growth	Multiple new measurement entries can be added simultaneously from different measuring sites and are not limited to one per hour, so the growth is unpredictable.

Table 3: Data Catalog for the Air Quality dataset

taxi trip parquets	
Primary entity	One reported taxi trip
Primary key	The dataset has no reliable natural primary key
Join attributes	VendorID, RatecodeID, PULocationID, DOLocationID, tpep_pickup_datetime, tpep_dropoff_datetime
Temporal attributes	tpep_pickup_datetime, tpep_dropoff_datetime
Categorical attributes	VendorID, RatecodeID, store_and_fwd_flag, payment_type
Growth	Multiple new entries can be added simultaneously so the growth is unpredictable.

Table 4: Data Catalog for the Taxi Trips dataset

Task 2.

Task 2 concerns the design of a storage architecture for the datasets. The following design decisions were considered:

Lookup Tables: The Taxi Zone Lookup dataset should be treated as a lookup table, since it does not represent continuous event streams or temporal measurements and does not have any projected growth. It only provides mapping from LocationID used in taxi trips to detailed zone/location information.

Partitioning strategy: Partitioning can be used to help break a large dataset into smaller, more manageable pieces. Partitioning strategies do not fit uniformly across all different datasets, our datasets require **different partitioning strategies**. The taxi_trips dataset requires high-frequency partitioning by date due to its nature as a high-volume fact table generating millions of rows per month. It is possible to partition the air quality dataset by time, just like the Taxi Trips dataset, but the weather and taxi_zone lookup datasets might not be the best contenders for partitioning. **Partitioning can become harmful** when it creates too many small partitions, which can make the system slower. Each partition adds to the growing number of directories/files stored as well as the metadata that Spark has to manage. Partitioning by too specific or exact attributes, such as partitioning air_quality or taxi_trips by the exact timestamp time_utc, becomes harmful since it results in thousands of tiny partitions. Partitioning can also become harmful when the chosen datasets are too small, since it introduces overhead that outweighs any benefit of storing it normally. For these reasons, the taxi_zone and weather datasets **should not be partitioned**. The taxi_zone dataset contains only around 265 records total, so there are not enough entries to justify the fractioned tiny Parquet files and the metadata management overhead in Spark. The Weather dataset contains only one record per hour, which results in around 730 records a month or 8.760 records in a full year. Similarly, it does not generate enough entries to justify the overhead. **Managing volume increase:** If the total data volume increased by 20x, the partitioning choices should be revisited to make the datasets more manageable. The monthly taxi_trips and air_quality partitions would be too large and should instead be changed to daily partitioning. The Weather dataset would benefit from partitioning as well, so monthly partitioning could be introduced. The taxi_zone dataset, even if 20x larger, would still be quite static and therefore would not require partitioning. Some cleaning could also be done to the datasets, like reorganizing the structure and organization to fit the larger scale. For example, extracting or even dropping verbose text columns in air_quality, such as method_name, state_name and county_name, and keeping only the codes (method_code, state_code and county_code).

Task 3.

In our framework we have both components that are generic and reusable, and ones that are dataset-specific. The loading of datasets, schema validation, and

the standardization of column names are all generic and reusable components, they don't need to change regardless of the dataset added. On the other hand, the components that normalize the timestamps are all dataset-specific since each dataset uses a different timestamp format. Therefore we have to go and change each one separately. Another dataset-specific component is the one filtering out unknown locations in the taxi trips dataset.

The transformation rules are defined as separate functions in the code, each handling a specific case. We then reference these functions in the config dictionary, where we specify which dataset needs which transformation rules. This makes it easier to maintain since we only have to update the function or a single config entry if the requirements change.

For each dataset, we generate a metadata record containing its schema version, execution time, row count, and a timestamp. These records are appended to a shared delta table, so the metadata does not get overwritten each run but creates a full history of the runs over time.

Overall, this design reduces code duplication by extracting dataset-specific differences into config, while the components that load, validate, and store the datasets are written once and shared between them all. This means that if something breaks it only needs to be fixed in one place, and adding a new dataset just means adding a config entry rather than writing a whole new pipeline.

If we were to add 20 new datasets next year, it would mainly mean writing 20 new config entries. The main code changes needed for the existing functions would be to support new file formats, and we would likely need to add more dataset-specific components as new edge cases could appear.

Task 4.

We use a shared data representation across all the datasets. If a dataset includes time, we create a *time_utc* column with the type *timestamp_ntz*, which acts as the common join key across datasets. We standardize all column names to a snake case. Rows that are missing a primary key or have a duplicated primary key are dropped. If a normalized timestamp is null or a row contains outliers that are clearly invalid we filter them out.

Task 5.

The final integrated dataset combines taxi trips with geographic, weather, and air-quality data, providing contextual information for each trip while preserving the original trip records.

For geographic enrichment, the pickup and dropoff location IDs are matched separately against the taxi-zone lookup data. This produces the columns *pickup_zone*, *pickup_borough*, *dropoff_zone* and *dropoff_borough*.

The weather and air quality datasets provide hourly observations, so each trip was associated with the observations corresponding to its pickup hour, using

the shared UTC timestamps established earlier, which provides a consistent temporal join across the datasets.

Left joins are used for the contextual enrichment so that missing weather or air-quality observations do not cause taxi trips to be removed. Instead, the corresponding fields remain NULL. This preserves complete trip coverage while making missing contextual information explicit and consistent with the data representation used elsewhere in the pipeline.

This integration strategy has several limitations. First, hourly weather and air-quality observations are an approximation of the conditions at the exact pickup time and might not be representative of the conditions observed by the passenger at the exact time and location of pickup. Second, drop off conditions are not accounted for in this implementation. Finally, missing observations remain as NULL, leaving further consideration to the upcoming layers.

Conclusion

Through the design and implementation of our data integration platform, we have been introduced to the main concepts of data-intensive computation platforms.

Working with the data platform, we learned to prioritize reusable ingestion logic rather than source-specific transformations, which is an important practice when designing distributed systems. This simplifies maintenance and makes it easier to extend in the future as new datasets might be added.